

SPRAWOZDANIE

Zajęcia: Matematyka Konkretna

Prowadzący: prof. dr hab. inż. Vasyl Martsenyuk

Laboratorium Nr 1 Data 27.11.2025 Temat: "Zadanie liniowe RNN" Wariant 15	Jakub Janik Informatyka II stopień, stacjonarne, 2 semestr, gr. CB
--	---

1. Cel ćwiczenia:

Celem algorytmu DMD jest znalezienie macierzy A (macierzy liniowego operatora ewolucji w czasie), która najlepiej opisuje przejście z jednego stanu do drugiego w całym zbiorze danych, minimalizując błąd Frobeniusa.

2. Przebieg ćwiczenia:

Implementacja Algorytmu DMD

W celu zwiększenia stabilności numerycznej i efektywności obliczeniowej, zastosowano zredukowaną formę DMD, opartą na dekompozycji według wartości osobliwych (SVD) macierzy X

Warunki Początkowe

Do celów demonstracyjnych użyto syntetycznych danych testowych

- Wymiar stanu $n=3$.
- Liczba próbek czasowych $m=10$.
- Zredukowany rząd SVD $r=2$.

Macierz	Wymiar	Opis
X	$N \times m$ (3×10)	Macierz stanów t_k
X'	$N \times m$ (3×10)	Macierz stanów t_{k+1}

Kroki Algorytmu

1. SVD Macierzy X

Wykonano dekompozycję SVD macierzy X z obcięciem do rzędu $r=2$

2. Obliczenie Zredukowanej Macierzy $\bar{A}$

Obliczono zredukowaną macierz $\bar{A}$ (o wymiarze $r \times r$), która definiuje model liniowy dynamiki dla współrzędnych $\tilde{x}$

3. Kompozycja skalarna

Obliczono wartości własne Λ oraz wektory własne W macierzy $\bar{A}$

Wynik (Wartości Własne Λ): [4.76976342, 0.4444867]

4. Rekonstrukcja Niezredukowanych Wektorów Własnych

Zrekonstruowano niezredukowane wektory własne (tryby DMD) Φ , które reprezentują dynamiczne tryby układu

Rozmiar macierzy Phi (Tryby DMD): (3, 2)

Kod źródłowy:

```
import numpy as np

# Przykładowe dane do demonstracji algorytmu DMD.
n = 3
m = 10
r = 2

# Tworzenie losowych danych
np.random.seed(42)
X = np.random.rand(n, m)
X_prime = X @ np.random.rand(m, m) * 1.05

print(f"Rozmiar macierzy X: {X.shape}")
print(f"Rozmiar macierzy X': {X_prime.shape}")

X_dagger = np.linalg.pinv(X) # Obliczenie pseudoodwrotności X
A_approx = X_prime @ X_dagger
print("\n--- Podstawowe Obliczenie Macierzy A ---")
print(f"Rozmiar macierzy A_approx: {A_approx.shape}")

# Obliczenie SVD macierzy X z rzędem r
U, Sigma, Vh = np.linalg.svd(X, full_matrices=False)

# Redukcja do rzędu r
U_tilde = U[:, :r]
Sigma_tilde = np.diag(Sigma[:r])
V_tilde = Vh[:r, :].T
```

```

# Obliczenie odwrotności zredukowanej macierzy wartości osobliwych
Sigma_tilde_inv = np.linalg.inv(Sigma_tilde)

# Obliczenie zredukowanej macierzy A_tilde
A_tilde = U_tilde.T @ X_prime @ V_tilde @ Sigma_tilde_inv

print("\n--- Obliczenie Zredukowanej Macierzy A_tilde (DMD) ---")
print(f"Rozmiar macierzy A_tilde: {A_tilde.shape}")

# Obliczenie kompozycji spektralnej A_tilde
Lambda_diag, W = np.linalg.eig(A_tilde)

print("\n--- Kompozycja Spektralna A_tilde ---")
print(f"Wartości własne (Lambda): {Lambda_diag}")

# Rekonstrukcja niezredukowanych wektorów własnych (tryby DMD)
Phi = X_prime @ V_tilde @ Sigma_tilde_inv @ W

print("\n--- Rekonstrukcja Niezredukowanych Wektorów Własnych (Phi) ---")
print(f"Rozmiar macierzy Phi (Tryby DMD): {Phi.shape}")

```

Wyniki:

```

Rozmiar macierzy X: (3, 10)
Rozmiar macierzy X': (3, 10)

--- Podstawowe Obliczenie Macierzy A ---
Rozmiar macierzy A_approx: (3, 3)

--- Obliczenie Zredukowanej Macierzy A_tilde (DMD) ---
Rozmiar macierzy A_tilde: (2, 2)

--- Kompozycja Spektralna A_tilde ---
Wartości własne (Lambda): [4.76976342 0.4444867 ]

--- Rekonstrukcja Niezredukowanych Wektorów Własnych (Phi) ---
Rozmiar macierzy Phi (Tryby DMD): (3, 2)

```

Link do repozytorium: <https://github.com/Uczelniane/MK.git>

3. Wnioski

Algorytm DMD został zaimplementowany pomyślnie, dostarczając zredukowany model dynamiki $\bar{A}$ i tryby Φ , a także podstawowe obliczenie macierzy A